Pràctica 2: Zeros de Funcions i Òrbites de Lyapunov

Mètodes Numèrics – Curs 2024-2025

Oriol Antoja i Albert Cortada
1711151 i 1702580

11 de maig de 2025

Resum

Aquesta memòria descriu la implementació i validació d'una llibreria genèrica de cerca de zeros, basada en els mètodes de la bisecció i de Newton, i la seva aplicació al *Restricted Three-Body Problem* (RTBP) per obtenir òrbites de Lyapunov planes al voltant del punt d'equilibri L_1 del sistema Terra–Lluna.

1 Introducció i objectius

El *Restricted Three-Body Problem* (RTBP) descriu el moviment d'un cos de massa negligible sotmès a dos primaris (Terra i Lluna) que orbiten circularment el seu centre de masses. En aquest context, les òrbites de Lyapunov planes al voltant del punt de Lagrange L_1 són un recurs clau per a missions d'estacionament, maniobres de transferència i dinàmica de captures gravitatòries.

Si prenem un sistema de coordenades que gira amb els primaris, trobar aquestes òrbites requereix resoldre l'equació d'una sola variable $u_f(x) = 0$, on u_f és la component x de la velocitat de la partícula al primer tall amb l'eix x. Aquesta pràctica té per objectiu dissenyar i validar una llibreria de cerca de zeros i aplicar-la al RTBP per obtenir una òrbita de Lyapunov plana.

Objectius concrets

1. Implementar els mètodes de bisecció i Newton a la llibreria `zeros.c`.
2. Verificar, mitjançant proves pròpies, el comportament dels dos algorismes.
3. Desenvolupar dues utilitats de consola:
   - `rtbp_lyap_csig`: genera la parella $(x, u_f(x))$ en un interval donat.
   - `rtbp_lyap_ref`: combina bisecció i Newton per trobar un zero de u_f.

2 Comentaris sobre la implementació del software demanat.

Per a la implementació del software, hem seguit el guió proposat per en Josep Maria Mondelo. Concretament, els algoritmes utilitzats per als mètodes de bisecció i Newton són exactament els descrits en el guió.

3 Manual del software

3.1 Biblioteques

3.1.1 `zeros.c`

La biblioteca `zeros.c` inclou dues funcions que implementen els mètodes de biseccio i Newton, respectivament, per trobar zeros d'una funció.

```
void biseccio(double a, double b, double tol,
              double *p, double *af, double *bf,
              double (*f)(double, void *),
              void *prm, int ixrr);
```

- `a` – extrem esquerre inicial de l'interval $[a, b]$.
- `b` – extrem dret inicial; cal que $f(a)\, f(b) < 0$.
- `tol` – tolerància per aturar-se quan $(b - a)/2 \leq$ `tol`.
- `p` – punter on s'escriu l'arrel aproximada.
- `af` – (si `!= NULL`) valor final d'`a` a la sortida.
- `bf` – (si `!= NULL`) valor final de `b` a la sortida.
- `f` – punter a la funció $f(x, \texttt{prm})$.
- `prm` – punter amb paràmetres necessaris per `f`.
- `ixrr` – nivell de xerrada: 0 silenciós, > 0 informa de com evoluciona el mètode per `stderr`.

```
double newton(double x0, double tolf, double tolx, int maxit,
              void (*fdf)(double, double *, double *, void *),
              void *prm, int ixrr, int *iret);
```

- `x0` – aproximació inicial de l'arrel.
- `tolf` – tolerància sobre $|f(x)|$ per a la parada.
- `tolx` – tolerància sobre el pas $|x_{k+1} - x_k|$.
- `maxit` – nombre màxim d'iteracions permeses.
- `fdf` – punter a funció que retorna $f(x)$ i $f'(x)$ via dos punters.
- `prm` – punter amb paràmetres necessaris per a `fdf`.
- `ixrr` – nivell de xerrada: 0 silenciós, > 0 informa de com evoluciona el mètode per `stderr`.
- `iret` – punter on s'escriu el nombre d'iteracions (-1 si no hi ha convergència).

3.2 Utilitats

3.2.1 `rtbp_lyap_csig`

Compilació

```
make rtbp_lyap_csig
```

Ús

```
./rtbp_lyap_csig c isgn x0 xf n
```

- `c` – constant de Jacobi.
- `isgn` – signe inicial de v (± 1).
- `x0` – extrem esquerre de l'interval de sondatge.
- `xf` – extrem dret de l'interval de sondatge.
- `n` – nombre de subdivisions (`n+1` punts avaluats).

Funció Per cada $x_i = x_0 + i(x_f - x_0)/n, 0 \leq i \leq n$, imprimeix a `stdout` les parelles $(x\ ,\ u_f)$. D'aquesta manera, podem indentificar un interval prou petit on u_f canvia de signe.

3.2.2 `rtbp_lyap_ref`

Compilació

```
make rtbp_lyap_ref
```

Ús

```
./rtbp_lyap_ref c isgn xa xb tolbis tolfnwt maxitnwt ixrr
```

- `c` – constant de Jacobi.
- `isgn` – signe inicial de v (± 1).
- `xa`, `xb` – extrems de l'interval inicial.
- `tolbis` – tolerància de la fase de bisecció.
- `tolfnwt` – tolerància per a $|u_f(x)|$ a Newton.
- `maxitnwt` – nombre màxim d'iteracions de Newton.
- `ixrr` – nivell de xerrada (0 silenciós, > 0 traça).

Funció 1) Aplica bisecció en $[x_a, x_b]$ fins que el punt mitg del interval és $\leq$ `tolbis`; 2) passa a Newton fins que $|u_f(x)| \leq$ `tolfnwt` o es supera `maxitnwt`. Imprimeix a `stdout` l'arrel refinada x (condició inicial que genera l'òrbita de Lyapunov).

4 Proves realitzades

Per tal de validar la biblioteca `zeros.c`, hem realitzat proves a les dues funcions que conté.

4.1 Proves del mètode de la bisecció

La utilitat `proves_bis_exp.c` cerca l'arrel de $f(x) = e^x - 2$ ($\alpha = \ln 2 \approx 0{,}6931$) a l'interval inicial $[a, b] = [0{,}5,\ 1]$ mitjançant `biseccio()`. La taula 1 mostra el nombre d'iteracions real (n_{exp}) i el teòric mínim

$$n_{\text{teo}} = \left\lceil \log_2\!\left(\frac{b-a}{\texttt{tol}}\right) \right\rceil,$$

que garanteix $(b-a)/2^{\,n} \leq \texttt{tol}$.

tol	n_{exp}	n_{teo}	$\lvert p - \alpha \rvert$
0.50	0	0	1.27×10^{-1}
0.25	1	1	7.65×10^{-2}
0.125	2	2	2.53×10^{-2}
0.0625	3	3	1.26×10^{-2}
0.03125	4	4	6.31×10^{-3}
0.015625	5	5	3.15×10^{-3}

Taula 1: Convergència de `biseccio()` amb diferents toleràncies.

Observacions. Quan reduïm `tol` a la meitat, n_{teo} augmenta exactament en una unitat, perquè la bisecció divideix l'error per 2 a cada pas (convergència lineal). Els resultats empírics coincideixen amb la predicció teòrica, validant la implementació.

4.2 Proves del mètode de Newton

La utilitat `proves_nwt_exp.c` aplica `newton()` a la mateixa funció $f(x) = e^x - 2$. La taula 2 presenta cinc jocs de paràmetres representatius (`x0`, `tolf`, `tolx`, `maxit`) i el comportament observat.

$\mathbf{x_0}$	**tolf**	**tolx**	**maxit**	n_{exp}	**Criteri parada**
2	10^{-8}	10^{-8}	10	6	$\lvert f \rvert \leq \texttt{tolf}$
2	10^{-8}	0	10	6	$\lvert f \rvert \leq \texttt{tolf}$
2	0	10^{-8}	10	6	$\lvert x_{k+1} - x_k \rvert \leq \texttt{tolx}$
10	10^{-12}	10^{-8}	10	10	*No convergeix*
10	10^{-12}	10^{-8}	20	15	$\lvert f \rvert \leq \texttt{tolf}$

Taula 2: Comportament de `newton()` amb diversos paràmetres.

Observacions.

- *Aproximació inicial propera ($x_0 = 2$).* Newton té convergència quadràtica i arriba a α en només 6 iteracions. Canviar el criteri de parada (`tolf` vs `tolx`) no afecta el nombre d'iteracions perquè tots dos s'assoleixen simultàniament.
- *Aproximació inicial llunyana ($x_0 = 10$).* Per $f(x) = e^x - 2$, la iteració és $x_{k+1} = x_k - 1 + \frac{2}{e^{x_k}} \approx x_k - 1$ mentre $x_k \gg \alpha$, de manera que la convergència és lineal lenta (resta ≈ 1 en cada pas). Amb `maxit`=10 no hi ha prou passos per entrar a la zona de convergència i l'algorisme s'atura per límit d'iteracions. Permetre `maxit`=20 dóna temps suficient (15 iteracions) per arribar a la tolerància sobre $\lvert f \rvert$.

5 Experiments numèrics

Amb l'objectiu de determinar el valor de x que dóna lloc a una òrbita de Lyapunov, hem representat la gràfica de u en funció de x. Aquest procediment ens permet identificar un canvi de signe en u, a partir del qual podem determinar un interval adequat per aplicar el mètode de la bisecció.
Per tal de generar les dades necessàries, hem executat la següent crida a la utilitat `rtbp_lyap_csig.c`:

```
./rtbp_lyap_csig 3.196255396314419 -1 -0.830 -0.829 100 > u.txt
```

El resultat ha sigut el següent:

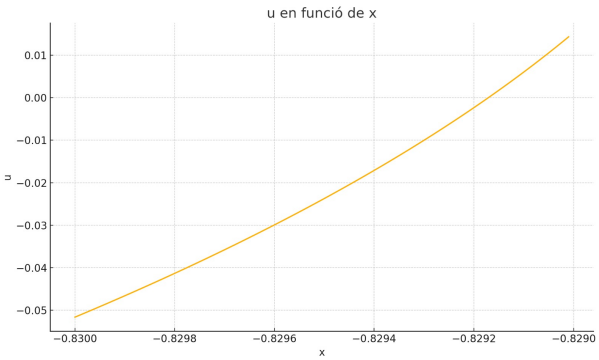


Figura 1: Representació de u en funció de x.

Finalment, hem grafitat la corresponent òrbita de Lyapunov plana propagant la condició inicial obtinguda amb la utilitat `rtbp_lyap_ref`. Aquesta condició inicial ha estat utilitzada com a entrada per a la rutina `proptraj()`, amb la qual hem generat la trajectòria fins al segon tall amb l'eix x.
La figura següent mostra el resultat obtingut:

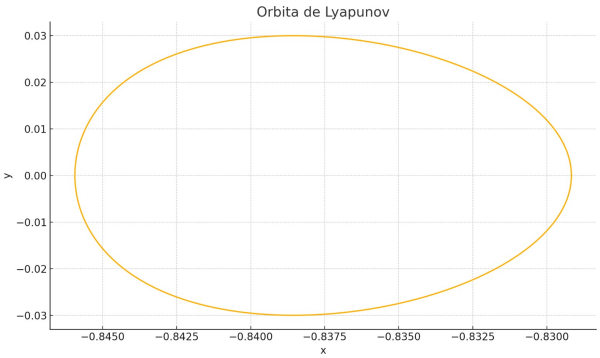


Figura 2: Òrbita de Lyapunov.